GenericAgent: A Self-Evolving LLM Agent via Contextual Information Density Maximization(V1.0)

Advantage AI Agent Laboratory

See Author Contributions (Section 6) for a full author list.

Abstract

Long-horizon Large Language Model (LLM) agents are fundamentally bottlenecked by context: as interactions extend, useful information is overwhelmed by accumulated noise, undermining reasoning quality and operational efficiency. We present GenericAgent (GA), a general-purpose agent system built around **context information density maximization**. To operationalize this principle, GA integrates four core mechanisms: a minimal atomic tool set that enforces essential interactions, hierarchical on-demand memory that retrieves context by relevance, code-driven self-evolution that distills past experiences into reusable capabilities, and selective browser extraction that compresses unstructured web content into task-relevant representations. These mechanisms continuously suppress low-value context and preserve high-signal information during long-horizon execution. We conduct extensive experiments of GA across multiple evaluation dimensions, including task success rate, efficiency, tool usage, memory behavior, and web interaction, enabling a comprehensive comparison of long-horizon execution capability. Results show that GA consistently improves the efficiency–performance trade-off, achieving higher task success at substantially lower token and interaction cost. This advantage persists across repeated runs, indicating stable, cost-efficient, and scalable agent behavior. We release GA as an open-source agent system at https://github.com/lsdefine/GenericAgent.

1 Introduction

The recent emergence of agentic systems such as Claude Code [1] and OpenClaw [2] signals a broader shift in the role of large language models (LLMs). Instead of serving solely as passive text generators, LLMs are increasingly deployed as goal-directed agents that operate through terminals, file systems, browsers, and external tools. This transition substantially expands their functional scope, while introducing new systems-level challenges. In long-horizon tasks, success depends jointly on the reasoning capability of the underlying model and the system's ability to manage context, retrieve relevant information, and accumulate useful experience over time. These requirements give rise to two fundamental challenges.

The first challenge is **context explosion**. As the agent interacts with the environment, the prompt grows beyond the initial user request. It accumulates tool definitions, retrieved memory, intermediate observations, prior traces, and raw web content. While these components support execution, they are not equally relevant to the next step and even compete with key instructions and evidence. This leads to redundant history, irrelevant memory, and poorly structured compression, which dilute attention and increase errors such as missed constraints, state confusion, and hallucinated facts [3–5].

The second challenge is **how to enable effective continual self-evolution**. Without mechanisms for distilling experience, agents fail to improve from repeated interactions in long-horizon environments. They repeatedly

fall into similar failure states, leading to unnecessary token consumption without capability gains. Therefore, the agent system enters a stagnation loop where performance remains unchanged despite continued interaction.

To this end, we propose **GenericAgent (GA)**, a self-evolving LLM agent system built around a fundamental principle: **maximizing contextual information density**. The core view is that agent performance is not determined by context length alone, but by how much decision-relevant information is introduced, maintained, and updated within the limited context budget.

GA realizes this principle through four mechanisms that operate across the lifecycle of contextual information. (1) A minimal atomic tool set reduces persistent tool overhead, preventing low-value interface information from occupying the context before task execution. (2) A hierarchical memory mechanism selectively retains only verified and task-relevant knowledge, shaping what information persists over time. (3) A reflection-driven self-evolution pipeline compresses interaction trajectories into reusable Standard Operating Procedures (SOPs), code, and skills, progressively transforming experience into compact and structured capability. (4) A structured browser extraction module converts noisy web pages into dense, execution-oriented observations before they enter the context. Together, these mechanisms continuously reshape the context from a passive accumulation of information into a high-density, decision-centric representation.

We evaluate GA at both the task and system levels. Across multiple benchmarks, GA achieves strong task completion performance while consistently reaching a superior trade-off between completion and token cost. Beyond performance alone, GA demonstrates systematic efficiency gains under repeated execution, as interaction traces are progressively distilled into reusable SOPs and code.

Our main contributions are as follows:

- We identify **context information density maximization** as a key design principle for long-horizon LLM agents, where the central challenge is to efficiently preserve and organize decision-relevant information under a fixed context budget.
- We propose **GenericAgent (GA)**, a general-purpose agent architecture that operationalizes this principle through four components: a minimal atomic tool set, hierarchical memory with on-demand retrieval, a reflection-driven self-evolution pipeline, and structured browser extraction for compact web observations.
- We introduce a **reflection-driven self-evolution mechanism** that summarizes execution traces into reusable SOPs, compiles stable procedures into code, and stores them as skills for later reuse.
- We conduct a comprehensive evaluation across task completion, token efficiency, self-evolution, tool use, memory, and web interaction, comparing GA with representative agent systems.

2 GenericAgent

2.1 Design Principles

Context information density is the core design constraint for LLM-based agents.

The central challenge in LLM-based agents is not merely context length, but context quality. Recent studies reveal three compounding failure modes that constrain how effectively models can use their input. First, models exhibit strong positional bias when processing long sequences: relevant information placed in the middle of the context is significantly harder to retrieve than information near the beginning or end [**?**]. Second, irrelevant content does not simply go unused; it can actively degrade performance by diverting attention away from decision-critical evidence [**?**]. Third, the effective context length of LLMs falls far short of their nominal window size, meaning that part of the provided context is functionally invisible during generation [**?**].

These three phenomena reinforce one another in practice. As context grows longer, positional bias makes middle-positioned evidence harder to retrieve, irrelevant content competes for the model's limited effective attention, and the declining ratio of effective to nominal context length means that an increasing fraction of the prompt is simply wasted. The result is that, beyond a certain point, adding more context does not improve performance and may instead reduce it.

This has direct implications for agent system design. Existing agent frameworks that rely on monolithic prompt assembly often devote a disproportionate share of the effective context budget to scaffolding, control text, and marginally relevant interaction history. Rather than exposing only decision-relevant information, such strategies displace the evidence most needed for the current step, leaving the agent with more context in volume but less context in utility, while still increasing inference cost.

From the perspective of context engineering, this problem is best understood in terms of two primary requirements and one secondary representational constraint:

- **Completeness.** All information required for the current decision must be explicitly present in the context, preventing the model from relying on implicit assumptions or hallucinated completions.
- **Conciseness.** Irrelevant and redundant information must be eliminated so that attention remains focused on decision-critical signals.
- **Naturalness.** The context should remain reasonably natural and semantically legible, especially when aggressive compression or artificial encodings make the representation harder for the model to use. This is an important but secondary constraint: in most agent settings, the dominant failure mode is not lack of natural phrasing itself, but the loss of completeness or conciseness.

We argue that completeness and conciseness define the primary design space for context quality in LLM-based agents. Completeness ensures that the model has the information it needs, while conciseness ensures that this information is not diluted by distracting or low-value content. Naturalness still matters, but mainly as a constraint on how information is represented rather than as the main bottleneck in most agent failures. It helps avoid brittle encodings, overly telegraphic summaries, and artificial formats that the model may parse less reliably. Other commonly discussed dimensions can largely be understood within this framework. For instance, recency serves both completeness, because the latest state is often necessary for the current decision, and conciseness, because outdated information is typically no longer relevant. Relevance, rather than being an independent axis, is better understood as a cross-cutting concern: both completeness and conciseness presuppose a judgment of what matters, the former in deciding what must be included and the latter in deciding what should be removed.

Critically, the main structural tension is between completeness and conciseness. This is not merely a consequence of finite context budgets. Even if the context window were unbounded, a structural tension would remain. The tension is structural rather than merely budgetary for at least three reasons:

- Including more potentially relevant information improves completeness but weakens conciseness.
- Summarization and compression improve conciseness but risk omitting details needed for completeness.
- Naturalness can sharpen this trade-off in some cases, because highly compressed or artificial representations may be harder for the model to interpret, but this is typically a secondary effect rather than the dominant constraint.

The finite effective context window [?] sharpens this tension into a hard practical constraint, but the underlying conflict is structural rather than merely resource-driven. In this sense, context engineering is best viewed not as an equal three-way trilemma, but as a constrained optimization problem centered on balancing completeness and conciseness, with naturalness acting as a supporting constraint on representation.

This principle serves as the guiding idea behind the design of GA. Rather than treating context as a passive byproduct of interaction history, GA systematically optimizes for information density at multiple layers of the system. Its memory hierarchy keeps a lightweight orientation layer visible while leaving richer facts and procedures to explicit reading, preserving conciseness without giving up access to deeper knowledge. Its browser interaction layer represents web pages through semantically structured observations rather than raw HTML, which improves readability while avoiding large amounts of low-value markup in context. Its self-evolution mechanism provides a way to consolidate successful experience into reusable memory and procedures, helping later tasks begin from more compact and task-relevant context. The following subsections describe how each mechanism instantiates this principle in detail.

2.1.1 Tool Minimality

GA is built on the premise that general-purpose agency does not require tool proliferation: a small set of well-chosen primitives is sufficient, provided that those primitives are functionally complete for the interactions the agent must perform. Following this premise, **GA uses only nine atomic tools while retaining broad task coverage** across file operations, code execution, web interaction, and system-level actions. This design is enabled by two supporting properties: **atomicity**, which constrains each tool to an irreducible primitive capability, and **compositional generalization**, which allows complex behaviors to be realized through sequences of such primitives. Together, these properties make a small tool set sufficient in practice: the agent does not need a separate interface for each task, only primitives that can be recombined to realize it. Table **??** summarizes the complete GA tool set.

Why not simply add more specialized tools? Because tool proliferation introduces system-level costs at two levels:

- **At the prompt level**, each additional tool enlarges the tool schema and associated instructions injected into the context; in multi-turn interaction, this overhead accumulates and occupies effective context budget that should be reserved for task-relevant information.
- **At the policy level**, each additional tool enlarges the action space and increases ambiguity in tool selection. This makes planning more brittle, weakens the stability of tool-use patterns, and increases the likelihood of execution errors and unnecessary retries.

Tool minimality is therefore not merely an austerity choice; it is a better operating point that reduces both prompt overhead and decision complexity.

Crucially, GA obtains capability through composition rather than tool enumeration. Its nine tools are designed as reusable primitives, not task-specific interfaces. File workflows emerge from combinations of `file_read`, `file_patch`, and `file_write`; computation and transformation are handled through the general-purpose executor `code_run`; and web tasks are realized through the perception–action pair of `web_scan` and `web_execute_js`. The remaining tools support the agent's long-horizon control loop: `ask_user` acquires missing information, `update_working_checkpoint` maintains short-term task continuity, and `start_long_term_update` consolidates reusable experience into persistent memory. Thus, GA does not need one tool per task. Broad capability arises from composing a small set of functionally complete primitives. Tool minimality is therefore not a restriction, but the mechanism by which GA preserves general-purpose capability while reducing interaction overhead.

2.1.2 Hierarchical Memory

For a general-purpose agent, the main memory challenge lies less in storage itself than in controlling how much information remains in the active context. In monolithic designs, prior interactions, intermediate states, instructions, and execution traces tend to accumulate as the task unfolds. Over long horizons, this causes the default prompt footprint to grow and consume context budget that is also needed for current reasoning and action. GA is designed around this tension: **rather than treating all retained information as prompt-resident by default, it keeps only a small always-on layer in view and accesses deeper memory only when needed.**

This leads to a layered memory structure in which the always-on layer is intentionally small. GA keeps an L1 memory index in the system prompt as a compact source of orientation: it records where stable resources are located and which global constraints should remain visible, but it does not carry detailed factual or procedural content itself. Richer knowledge is placed deeper in the hierarchy. Stable cross-task facts are kept in a fact layer, while task-specific know-how is stored as SOPs or executable scripts. These resources remain available to the agent through on-demand access rather than through default inclusion in the active context.

Historical interaction data is handled in the same way. Raw sessions are moved out of the always-on layer, archived, and compressed, so that long-run accumulation does not automatically become default prompt growth. This keeps detailed history available without requiring it to occupy the active context during routine reasoning. In this sense, hierarchy in GA is not only an organizational device; it is what keeps the default prompt footprint small while preserving access to richer history and task knowledge when they are relevant.

GA also separates three memory functions that are often conflated in agent systems, and implements them through different mechanisms rather than a single undifferentiated memory store:

- **Working memory** is the task-local continuity buffer. It is updated explicitly through `update_working_checkpoint`, which stores compact task state such as the current objective, constraints, progress, and relevant SOP pointers. This state is then re-injected in subsequent turns together with recent interaction history, allowing the agent to preserve execution continuity across long tasks without reconstructing the full state each time.
- **Always-on memory** is a lightweight orientation layer, not the full persistent memory base. In the implementation, it is injected into the initial system prompt and periodically refreshed during long runs. Its default contents are the working directory, the fixed memory layout, and the compact L1 insight index. This design keeps durable guidance continuously available while avoiding the context cost of loading full L2 factual memory or L3 SOP files by default.
- **Long-term memory** is handled as an explicit post-task consolidation process. Rather than being appended to the prompt at every step, it is written back only when `start_long_term_update` is triggered after useful experience has been validated. The update procedure follows the memory-management SOP and performs minimal writes to persistent factual memory (L2) or procedural memory (L3), so that only stable and verified knowledge is retained.

These layers therefore differ in lifetime, injection rule, update condition, and context footprint.

The resulting benefit is tighter control over the default prompt footprint. As interaction history grows, memory is no longer carried forward by default simply because more turns have occurred. Instead, the always-on layer remains small, while bulkier history and task knowledge stay in deeper storage and enter the active context through on-demand access. This preserves more of the context window for current reasoning and action. Better retrieval focus and stronger persistence follow from the same structure, but they are secondary to this more basic effect: memory does not steadily displace the active-context budget needed for the task at hand.

2.1.3 Self-Evolution as Experience Consolidation

If hierarchical memory controls how past information is retained and accessed, self-evolution concerns whether past execution can remain useful beyond the episode in which it occurred.

A general-purpose agent cannot assume that all useful task knowledge is already present in its initial prompt or latent prior. In open environments, many important regularities are local: they concern a specific user's file system, workflow conventions, browser routines, external services, and recurring task patterns. If such knowledge is never retained beyond the current episode, similar tasks may require repeated rediscovery. **GA is designed to address this problem by supporting experience consolidation across tasks, rather than treating each task as a fully isolated interaction.**

This claim should be stated carefully. Self-evolution in GA is not parameter learning, nor does it imply that every completed task is automatically transformed into persistent knowledge. The underlying model is not updated after each episode. Instead, GA provides a layered external memory structure, explicit update entry points for task-state and long-term memory, and memory-management guidance for deciding what should be retained after execution. In this sense, what can change over time is not the model itself, but the informational environment in which the model operates.

This design is relevant to the context-quality trilemma introduced earlier. Without any mechanism for retaining validated experience, the agent often faces a recurring dilemma on repeated tasks: either replay a longer exploratory process, which weakens conciseness, or act from a shorter but underspecified prompt, which weakens completeness. GA does not eliminate this tension, but it creates a way to reduce it. When useful experience is explicitly consolidated into memory or reusable task procedures, later tasks may begin from a better starting point than pure rediscovery. **The role of self-evolution in GA is therefore best understood as creating the conditions under which prior successful experience can be reused in a more compact and task-relevant form.**

Just as importantly, GA does not treat all past interaction as equally worth preserving. Raw execution traces contain failed branches, tentative hypotheses, redundant probing, and one-off detours that are useful during search but often undesirable as persistent context. For this reason, self-evolution in GA is better characterized as **selective consolidation** than as unrestricted memory accumulation. The system provides explicit pathways for updating working state and writing longer-term memory, while its memory-management guidance biases retention toward information grounded in successful execution and away from volatile, weakly verified, or purely situational content. These criteria should be understood as consolidation guidance and operator guidance, not as a claim that the framework already enforces a complete automatic test of validity, stability, or future reuse.

This selective view also clarifies what kind of compression is at stake. GA does not compress all experience into a single canonical representation. Instead, different retained materials occupy different layers of the memory hierarchy. A lightweight insight layer remains visible as a compact orientation signal, while richer factual knowledge, reusable task procedures, and archived session history remain in deeper storage. Self-evolution therefore does not mean that every past task is reduced to one uniform reusable object. Rather, GA provides a concrete path for consolidation through explicit memory-update actions, a layered retention structure, and memory-management guidance, allowing parts of past execution to be preserved in forms that are smaller, more targeted, and potentially more reusable than the original trajectory.

This point also links self-evolution to the two mechanisms discussed above. The relation to tool minimality should be understood carefully. Tool minimality does not by itself produce self-evolution, nor does it establish that fewer tools necessarily yield better consolidation. In GA, however, retained know-how is naturally expressed less as new task-specific interfaces than as reusable ways of sequencing and combining a small set of general primitives. Hierarchical memory, in turn, keeps retained experience from becoming default prompt clutter. In GA, a lightweight orientation layer remains continuously visible, while richer factual and procedural materials are accessed through explicit reading when the agent chooses to inspect them. **Tool minimality therefore constrains how action is expressed, hierarchical memory governs what remains visible by default, and self-evolution refers to the persistence of selected experience across tasks within that architecture.**

Under this interpretation, self-evolution should be understood not as a guaranteed automatic skill-growth pipeline, but as an architectural capacity for cross-task adaptation. GA supplies the memory layers, update interfaces, and consolidation guidance through which successful experience may persist beyond the episode in which it was acquired. When such retained experience is later read and applied effectively, it can reduce repeated exploratory effort and support better adaptation to a user's local environment. The main claim, however, is architectural rather than empirical: **GA is built to support experience-driven improvement over time through externalized memory and reusable task procedures, even though the extent and consistency of that improvement depend on what is consolidated, what is later read, and how it is applied in subsequent tasks.**

2.1.4 Context Truncation and Compression

GenericAgent does not preserve the full interaction history verbatim in the active prompt. Instead, it maintains a compact working context under an approximate character-length budget and progressively reduces older material when the accumulated prompt becomes too large. This mechanism is implemented at the prompt-construction level and should not be interpreted as precise token-level window control.

The implementation combines three distinct layers of context management. The first is a *turn-summary layer*. At the end of each turn, the framework appends one short summary entry to its internal summary list: it uses the model-provided `<summary>` field when present, and otherwise falls back to a simple summary derived from the physical action of that turn. In subsequent turns, the working prompt injects only the most recent 20 such entries. Thus, what is retained here is a bounded list of one-line turn summaries rather than the last 20 turns in full.

The second is a *task-state layer*. Through the explicit tool `update_working_checkpoint`, the agent may write compact task-critical notes such as user constraints, current findings, file paths, or next-step guidance into `key_info` and related fields. Once written, these fields are automatically injected into later working prompts.

The checkpoint is therefore explicitly updated by the agent and then automatically carried forward; it is not a background mechanism that autonomously rewrites task state on every turn.

The third is a *raw-history maintenance layer*. Older messages are compressed by format-aware, length-driven truncation rather than by semantic importance estimation. In particular, older `<thinking>`, `<tool_use>`, and `<tool_result>` blocks may be shortened by retaining only abbreviated head-and-tail segments, while stale embedded `<history>` and `<key_info>` regions may be replaced with placeholders such as `[...]` . Accordingly, this mechanism should be understood as tagged-content shortening under length constraints, not as selective preservation of semantically important reasoning content.

When such compression is still insufficient, the system trims the stored message history further. Once the serialized history exceeds the configured approximate budget, older messages are removed from the front, and trimming continues until the retained history falls back below a lower target threshold rather than merely touching the limit. During this process, the implementation also performs lightweight structural hygiene: the remaining history is realigned to begin at a user message, and an initial user message containing tool-result blocks may be rewritten into plain text to avoid leaving an orphaned tool-result fragment at the start of the retained context. This supports a narrow claim of basic structural validity preservation during trimming, but not a stronger claim of semantic reconstruction of prior history.

Related mechanisms also help limit prompt growth at specific tool boundaries. For example, some tool paths support file-based offloading of bulky outputs, and some returned content is shortened before reinsertion into the prompt. However, these should be treated only as tool-specific support for prompt-budget management, not as a general automatic externalization mechanism for all large intermediate artifacts.

Taken together, these mechanisms show that GenericAgent maintains a compact decision state without replaying the entire prior trajectory inside the active prompt. More specifically, it combines bounded injection of recent turn summaries, explicitly updated short-term task-state fields with automatic reinsertion, and length-driven compression and trimming of older history under an approximate character-length budget. These mechanisms are intended to make limited active context more sustainable during multi-step interaction. By themselves, however, they should be read as implementation-level prompt-budget management rather than as direct proof of long-horizon task success under a fixed small context window.

2.2 Overview of GenericAgent

GA is a general-purpose LLM agent system that provides automation over a local computing environment. It enables a compatible LLM backend to operate across browsers, terminals, file systems, keyboard and mouse inputs, screen-based perception, and mobile devices through configurable tool adapters. The system is model-agnostic, allowing the underlying reasoning engine (e.g., Claude, GPT, Gemini) to be replaced without affecting execution logic, tool interfaces, or memory architecture. The overall architecture is illustrated in Figure **??**.

GA is organized around a unified agent loop that connects the model and the environment. At each step, the system combines global memory with the current task specification to form an execution context. The LLM processes this context and produces either direct outputs or tool calls. Tool execution is delegated to external modules, and the results are returned as structured signals that update the system state. This loop defines a continuous interaction between reasoning and environment feedback, enabling tool-grounded iterative execution.

On top of this loop, GA supports two execution modes. **Interactive execution** handles user-initiated tasks such as code execution, information retrieval, and file operations. **Reflective execution** runs in the background and is triggered by schedules or system events. It is responsible for monitoring, memory consolidation, and experience distillation. Both modes share the same agent loop and memory system, differing only in initiation and output handling. To support long-horizon operation, GA defines explicit execution bounds that ensure controllability while preserving interruptibility and scope constraints.

After task completion, GA performs memory distillation. Execution traces are compressed into structured long-term representations and stored in shared memory. These representations are reused across both execution

modes, allowing accumulated experience to improve future execution through compression and reuse rather than unbounded computation.

2.3 Core Components of GenericAgent

Building on the principles in Section 2.1, we describe the core components of GA, covering the atomic tool set, hierarchical memory management, reflection-driven self-evolution, and structured-extraction-based browser interaction.

2.3.1 Minimal Atomic Toolset

GA exposes nine atomic tools, grouped into five capability classes. The **File Operations** class includes `file_read`, `file_patch`, and `file_write` for read, precise edit, and block write operations. The **Code Execution** class contains `code_run`, which runs Python or Bash in a controlled runtime. The **Web Interaction** class includes `web_scan` and `web_execute_js` for low-cost page inspection and precise browser actions. The **Memory Management** class includes `update_working_checkpoint` and `start_long_term_update` for short-term context maintenance and long-term memory distillation. The **Human-in-the-loop** class is `ask_user`, used when user decisions are required. Each tool maintains a single responsibility. These tools form a complete loop covering state observation, action execution, context preservation, and intervention requests.

GA adopts an authority–responsibility aligned permission model. Authority is strictly defined by the injected toolset, and GA cannot self-escalate beyond it. With read-only tools, GA can inspect and reason, but cannot modify any state; With patch tools, GA can edit text, subject to strict matching and parameter constraints. With execution and web-control tools, GA can trigger real system actions, while all interactions return structured outputs through the dispatcher, ensuring full traceability. The objective is not to grant unlimited capability, but to enforce controlled capability with aligned risk and clear accountability.

In terms of tool definition, GA represents tool capabilities as a verifiable schema contract and routes execution through a unified dispatcher. Each tool defines a name, a description, and a set of parameters described by JSON Schema, all under a consistent type function format. At runtime, this schema is injected into the interface, allowing GA to produce structured `tool_use` calls with explicit names and arguments. The dispatcher maps each call to a local executor and manages pre- and post-execution handling together with the returned results. This brings a clear separation between declaration, invocation, and execution. It does not need to know how tools are implemented; it only needs to follow a stable interface contract. GA then resolves each call against that contract and dispatches it to the concrete handler, separating call semantics from implementation details.

GA also specifically optimizes each atomic tool for decision efficiency and token economy. `file_read` supports segmented reads (`start`/`count`), keyword anchoring, and line-number output, so GA can inspect only relevant regions. `file_patch` enforces unique `old_content` matching and fails fast on zero or multiple matches, reducing silent multi-location edits. `web_scan` returns a simplified page view with tab metadata, which lowers full-DOM inspection cost. `web_execute_js` returns action results plus page-change observations, so many workflows proceed without another full scan. GA does not merely expose raw operations. It recasts real-world operations into low-ambiguity, verifiable, and cost-aware decision interfaces.

In summary, GA is not designed to make the model "do more." It is designed to define what can be done, constrain how it is done, and record what was done. Through atomic tool abstraction, schema contracts, unified dispatch, and structured receipts, GA aligns capability boundaries, risk boundaries, and accountability boundaries within a single operating framework.

2.3.2 Hierarchical Memory Architecture

GA's memory management mechanism does not directly append all historical information into the context. Instead, it organizes knowledge through hierarchical structuring, on-demand retrieval, and procedural consolidation. This design helps the agent stay coherent in long runs and turn validated experience into durable knowledge.

GA starts with a global meta-memory layer. This layer defines the memory map, core rules, and update boundaries. At task start, GA injects this layer first. It gives the model a shared frame of reference before execution. The model knows how memory is organized, what each layer is for, and how updates should be handled. This reduces arbitrary writes, historical misreads, and cross-task leakage.

Below the meta-memory, long-term memory is split into three layers with distinct roles. **(1)L1 is the index layer.** It stores compact pointers, including high-frequency entry points, keyword mappings, and a small set of hard constraints. Its main role is fast navigation. It enables efficient memory routing and ensures that relevant information can be quickly located. It works as a directory that guides GA toward appropriate SOPs, factual records, and core constraints for a given task type. **(2) L2 is the fact layer.** This layer stores verified and stable factual information that remains valid over long periods. Admission is strict. It includes environment paths, fixed configurations, persistent constraints, and system properties that can be repeatedly confirmed. Information is only written into L2 after it has been validated through execution and proven reusable across tasks. Transient states, one-time events, and unverified hypotheses are excluded. Updates to L2 are performed through small and traceable modifications rather than large-scale overwrites. This design prevents short-term noise from being stored as long-term facts. **(3) L3 is the SOP layer.** This layer stores reusable procedural knowledge. It includes task workflows, preconditions, key execution steps, common failure cases, and corresponding debugging or recovery strategies. Entries in L3 must come from real execution. They must also be validated across multiple tasks. Only knowledge that can directly guide error analysis or be reused in future tasks is included in this layer. These three layers form a continuous memory chain. GA first uses L1 for routing and memory localization. It then retrieves L2 to constrain reasoning with verified facts. Finally, it applies L3 to obtain actionable procedures for execution. This layered workflow improves both retrieval efficiency and execution reliability in long-horizon tasks.

During runtime, it uses an on-demand loading strategy. GA first injects only the minimal required information, including the meta-memory and the L1 index layer. Additional factual or procedural knowledge is retrieved only when needed during execution. This reduces irrelevant context noise and improves token efficiency. It also ensures that context budget is allocated to task-relevant information.

For long-term consolidation, GA uses a triggered commit mechanism instead of immediate writing. When information is identified as potentially valuable, it enters a validation stage. Its usefulness and reusability are checked before being committed. Only verified information is written into L2 or L3 through small, incremental updates. The L1 index is updated accordingly. This separation between execution and memory writing reduces incorrect updates and prevents memory bloat.

In addition to long-term memory, GA maintains a session-level working memory. It is treated as short-term operational cache and is not automatically promoted to long-term memory unless it satisfies the consolidation criteria. It mainly records the current goal, intermediate conclusions, key constraints, and next actions. This memory is continuously injected across turns to maintain task continuity. After each step, a compact summary is generated to form a rolling execution trace. This prevents degradation of context quality in long conversations.

2.3.3 Self-Evolution Capability in GA

The self-evolution capability of GA is realized through a fixed execution loop that continuously improves its behavior over time. In each iteration, GA first retrieves prior experience together with the current task state, then performs actions through external tools, and feeds the results back into the next decision step. Finally, strategies that are verified to be effective are distilled and stored as long-term experience. In this way, capability improvement is transformed from an implicit reasoning process into an explicit and continuously optimizable execution pipeline.

This loop is enabled by a clear separation between base tools and higher-level evolving capabilities. Base tools remain atomic and task-agnostic, covering only fundamental operations. Task-specific strategies are not embedded in these tools, but represented as reusable skills and SOP-like knowledge units. As a result, capability improvements do not require changes to tool implementations.

Memory is designed to support both decision making and stable execution over long tasks. At task startup,

Table 1 Task completion rate and token efficiency across the main agent benchmarks and RealFin-benchmark. Each block reports the original metrics used in the corresponding benchmark. Since the efficiency ratio is normalized with different token scales across benchmarks, its absolute values should only be compared within the same benchmark block.

Agent	Model	Accuracy	Input Tokens	Output Tokens	Total Tokens	Efficiency
			SOP-Bench			
GA	Claude Sonnet 4.6	100%	2.02M	53k	2.08M	0.48
OpenClaw	Claude Sonnet 4.6	100%	2.60M	40k	2.64M	0.38
Claude Code	Claude Sonnet 4.6	85%	1.23M	23k	1.25M	0.68
GA	Minimax M2.7	90%	893k	32k	924k	0.97
OpenClaw	Minimax M2.7	95%	2.91M	46k	2.96M	0.32
			Lifelong AgentBench			
GA	Claude Sonnet 4.6	100%	222k	20k	241k	4.15
OpenClaw	Claude Sonnet 4.6	70%	1.43M	21k	1.45M	0.48
Claude Code	Claude Sonnet 4.6	75%	800k	14k	814k	0.92
GA	Minimax M2.7	90%	400k	23k	423k	2.12
OpenClaw	Minimax M2.7	70%	1.20M	17k	1.22M	0.57
			RealFin-benchmark			
GA	Claude Sonnet 4.6	65%	102k	12k	114k	5.70
Claude Code	Claude Opus 4.6	60%	290k	17k	307k	1.95
Claude Code	Claude Sonnet 4.6	55%	226k	12k	238k	2.31
OpenClaw	Claude Sonnet 4.6	35%	249k	2k	115k	3.04
Codex	GPT-5.4	60%	838k	54k	892k	0.67

GA loads relevant prior experience to provide an initial plan and direction. During execution, each step stores a small summary of the current state, including key decisions, constraints, and progress. This allows GA to keep track of what has been done without storing the full history. As a result, the model does not need to re-derive the task context at every step, and it avoids drifting away from the original strategy in long-horizon execution.

To ensure reliable self-evolution, GA does not store raw execution traces directly. Instead, it only updates long-term memory at specific consolidation points. At these points, GA summarizes the task trajectory into useful and reusable knowledge. It keeps successful strategies and key decisions, and removes temporary or noisy information that is not stable or transferable. In addition, GA uses a fallback mechanism to correct the strategy when it detects repeated failures or unproductive loops. This prevents the system from reinforcing wrong behaviors.

2.4 Subagent

3 Evaluation

3.1 Task completion and token efficiency

In this section, we evaluate GA from five complementary perspectives. (1) We study task completion and token efficiency to measure overall capability and cost effectiveness across benchmarks. (2) We analyze tool-use efficiency to evaluate whether a minimal atomic-tool design can support complex tasks with lower interaction overhead. (3) We evaluate memory system effectiveness to test repeated-use improvement, Condensed memory, long-term fact retention, and context explosion prevention. (4) We examine self-evolution capability to test whether GA can compress successful trajectories into reusable SOPs and executable procedures. (5) We assess web browsing capability to examine GA's end-to-end browser interaction, multi-hop web search, and real-world web task performance.

Table 2 Source-level tool inventory overview of Claude Code, OpenClaw, and GA. We list representative native tools for Claude Code and OpenClaw and show the full atomic tool set of GA. The counts refer to source-level built-in tools/tool factories rather than the exact runtime-visible tool set, which may vary with configuration, feature flags, permissions, and plugin-based dynamic injection.

Claude Code	OpenClaw	GA
`AgentTool`	`browser`	`file_read`
`TaskOutputTool`	`canvas`	`file_write`
`BashTool`	`nodes`	`file_patch`
`GlobTool`	`cron`	`code_run`
`GrepTool`	`message`	`web_scan`
`FileReadTool`	`tts`	`web_execute_js`
`FileEditTool`	`gateway`	`ask_user`
`FileWriteTool`	`agents-list`	`update_working_`
`NotebookEditTool`	`sessions-list`	`checkpoint`
`WebFetchTool`	`sessions-history`	`start_long_`
...	...	`term_update`
53 source-level built-in tools: 20 base + 33 conditional.	*18 source-level tool factories; runtime may include plugins.*	*only 9 atomic tools*

3.1.1 Setup

We compare four representative agent systems, including GA, Claude Code, OpenClaw, and Codex. The backbone models include Claude Sonnet 4.6, Claude Opus 4.6, Minimax M2.7, and GPT-5.4.

Benchmark. We evaluate the systems on three complementary benchmarks. (1) **SOP-Bench** [6] tests multi-step SOP execution. (2) **Lifelong AgentBench** [7] tests sequential tasks with cross-task dependency. (3) **RealFin-benchmark** [8] tests realistic financial tasks that require domain understanding and reliable execution.

3.1.2 Results

GA maintains leading or near-leading task completion across different benchmarks. As shown in Table 1, this trend is consistent across SOP-Bench, Lifelong AgentBench, and RealFin-benchmark. On the Claude-based runs, GA achieves 100% accuracy on both SOP-Bench and Lifelong AgentBench, matching the best result on SOP-Bench and outperforming all baselines on Lifelong AgentBench. A similar pattern is observed on RealFin-benchmark, where GA attains the highest accuracy of 65%, exceeding Claude Code Opus and Codex at 60%, Claude Code Sonnet at 55%, and OpenClaw at 35%.

GA requires fewer tokens to complete tasks, especially in terms of input tokens. On SOP-Bench, GA uses 2.02M input tokens, which is lower than OpenClaw at 2.60M but higher than Claude Code at 1.23M, while still achieving 100% accuracy. In contrast, on Lifelong AgentBench, GA uses 222k input tokens, significantly lower than Claude Code at 800k and OpenClaw at 1.43M, while maintaining 100% accuracy. A similar pattern is observed on RealFin-benchmark.

GA achieves the strongest overall performance in task completion and efficiency across the main benchmarks. On both Lifelong AgentBench and RealFin-benchmark, GA achieves the strongest combined result in terms of task completion and token efficiency, reaching 100% accuracy with an efficiency ratio of 4.15 on Lifelong AgentBench and 65% accuracy with 114k total tokens and an efficiency ratio of 5.70 on RealFin-benchmark. On SOP-Bench, although Claude Code reaches a higher efficiency ratio of 0.68, its accuracy drops to 85%, whereas GA maintains 100% accuracy with an efficiency ratio of 0.48.

3.2 Tool-use efficiency

Table 3 Aggregated results on long-horizon complex tasks. This table reports the average outcomes over the long-horizon task set rather than expanding into per-task detail.

Agent	#Tasks	Success	Accounted Tokens	Time (s)	Requests	Tool Calls	Distinct Tools / Caps.
Claude Code	5	100.0%	537,413	320.8	32.6	22.6	4.4 / 3.8
GA	5	100.0%	188,829	220.8	11.0	12.8	3.6 / 3.6
OpenClaw	5	80.0%	633,101	183.1	15.0	16.6	4.2 / 3.2

3.2.1 Setup

Baseline. All compared systems use the same backbone model, Claude Sonnet 4.6. We evaluate three representative tool-use designs. (1) **GA**, which uses a minimal atomic-tool design based on a small set of general-purpose tools. (2) **Claude Code**, which serves as a strong baseline with a richer specialized tool ecosystem. (3) **OpenClaw**, which represents a broader multi-tool design with several specialized or environment-dependent capabilities.

Tool. Table 2 first gives a source-level inventory overview. Claude Code exposes 53 built-in tools at the source level (20 base tools and 33 conditional tools), while OpenClaw provides 18 source-level tool factories, with the runtime-visible set further varying by plugin injection and environment configuration. GA does not attempt to reproduce these full inventories one by one. Instead, it keeps only a small set of core atomic tools that cover the capabilities most frequently required in our evaluation tasks, and each of these capabilities has a clear counterpart in both Claude Code and OpenClaw. Appendix 1 provides both a capability-level alignment (Table 9) and representative task-level substitution examples (Table 10).

Benchmark. The benchmark contains two complementary task groups. (1) **Simple tool-generalization tasks**, which test whether baseline specialized tool capabilities can be reproduced through GA's atomic-tool composition. (2) **Long-horizon complex tasks**, which test whether the same minimal tool design remains effective on realistic multi-step workflows.

3.2.2 Atomic Tool Generalization

GA can solve complex long-horizon tasks through compositions of atomic tools rather than through a richer specialized tool inventory. In Table 3, GA reaches 100% success on the five long-horizon tasks, matching Claude Code and outperforming OpenClaw, which falls to 80%. This shows that a minimal tool set does not reduce the range of tasks that can be solved when the tools are composable in the right way. Appendix 1, especially Table 10, provides concrete substitution examples showing how several specialized baseline capabilities can be reconstructed through short atomic-tool compositions.

GA reduces token overhead while preserving task performance. Although it matches Claude Code on success, GA uses only 188,829 accounted tokens, which is 35.1% of Claude Code's 537,413 and 29.8% of OpenClaw's 633,101. The same pattern appears in the interaction statistics: GA reduces requests from 32.6 to 11.0 and tool calls from 22.6 to 12.8 relative to Claude Code, while OpenClaw remains both less stable and more expensive. These results indicate that GA's advantage comes from lower context and tool-process overhead, not from sacrificing success rate.

3.2.3 Tool Usage Distribution

Tool usage is highly concentrated in a few high-frequency tools, while the remaining long-tail tools are invoked only rarely. As shown in Figure 1, although Claude Code and OpenClaw expose dozens of built-in tools or tool factories, their actual execution traces are still dominated by a small subset of tools. In Claude Code, `AgentTool` alone accounts for 50.4% of calls, followed by `WebFetchTool` (22.1%), `FileReadTool` (10.6%), and `FileWriteTool` (8.9%), while many other tools appear only in the tail. OpenClaw shows a similar concentration pattern. This means that a substantial number of low-frequency tools still occupy prompt context and enlarge the action space even though they contribute little to actual execution. By comparison,

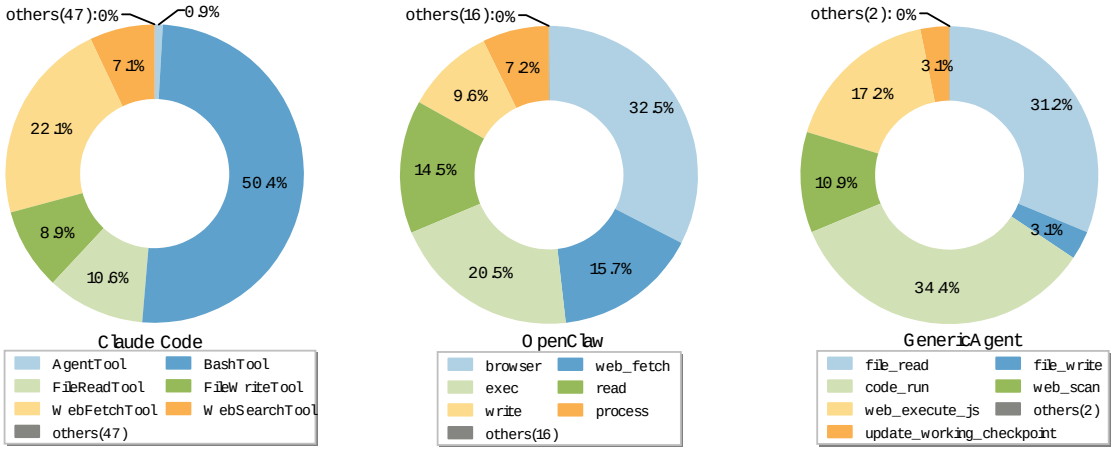

Figure 1 Tool usage distribution across Claude Code, OpenClaw, and GA. Each donut chart summarizes the proportion of tool calls attributed to each major tool category in the corresponding system.

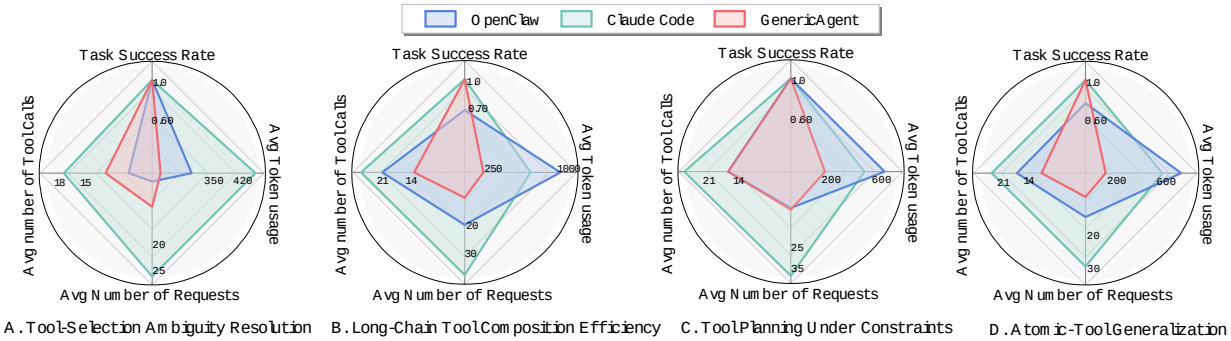

Figure 2 Dimension-wise radar comparison of tool-use efficiency. The four panels correspond to ambiguity resolution, long-chain composition, constrained planning, and atomic-tool generalization. Each radar chart compares Claude Code, OpenClaw, and GA on success, quality score, token cost, request count, and tool-call count. Token, request, and tool-call axes are inverted so that larger area consistently indicates better overall efficiency.

GA makes the active tool loop explicit through a compact atomic-tool set centered on `code_run`, `file_read`, `web_execute_js`, and `web_scan`, which is consistent with its lower request count, lower tool-call count, and lower token cost in Table 3.

3.2.4 Dimension-Wise Tool Efficiency Comparison

The minimal tool design remains effective across different task types. As shown in Figure 2, GA is the most balanced system across the four dimensions: it preserves strong task quality while keeping token, request, and tool-call overhead low. Claude Code behaves like a high-quality but high-overhead baseline, since it usually maintains success and quality but pays a larger cost through longer interaction traces and heavier tool usage. OpenClaw shows a different profile: some efficiency axes remain locally competitive, but its overall shape becomes less stable on the harder dimensions because success and quality begin to drop. Compared with both baselines, GA combines the stable quality profile of a strong system with a more efficient interaction pattern, which suggests that its advantage comes from disciplined tool abstraction and reusable capability composition rather than from exposing a larger number of specialized tools.

3.3 Memory System Effectiveness

To validate the effectiveness of GA's memory system, we design four experiments. (1) We test whether GA becomes more efficient through repeated use, (2) we examine whether Condensed memory is more

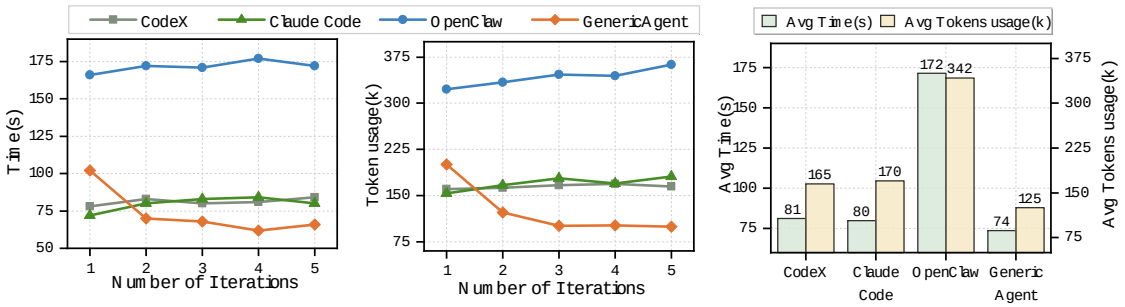

Figure 3 Operation time and token consumption across five repeated runs for different agents. While the baseline agents remain relatively stable over time, GA shows a clear efficiency convergence pattern, with both runtime and token cost decreasing substantially as task experience is accumulated and distilled into reusable memory.

effective than full or redundant memory, (3) we evaluate whether GA can support long-term factual memory without an additional embedding model or vector database, and (4) we study whether its hierarchical memory management can prevent memory explosion by retrieving memory only when it is actually needed.

3.3.1 Continuous Efficiency Improvement

Aiming to evaluate whether the Condensed memory framework enables GA to improve through continued real-world use, we focus on the agent's performance trajectory across repeated runs to assess gains in efficiency and reduced token consumption driven by accumulated experience. To this end, we compare CodeX, Claude Code, OpenClaw, and GA under the gpt-5.4. Each experimental run consists of downloading five different datasets from HuggingFace. To avoid contamination from residual context, every download task is conducted in a fresh session.

As shown in Figure 3, CodeX, Claude Code, and OpenClaw remained largely stable across runs, whereas GA improved markedly, with operation time decreasing from 102 seconds in the first run to approximately 66 seconds in later runs, and token consumption dropping from 200,439 to 100,000. This steady reduction indicates that **GA does not simply accumulate history, but converts task experience into reusable L3 SOPs that reduce repeated understanding, unnecessary reasoning, and decision overhead in later runs.** More importantly, the gain comes not from storing more content, but from preserving high-value information that the model does not already know and that directly affects behavior. Overall, the results show that GA becomes more efficient through continued use, which is precisely the intended effect of its memory system.

3.3.2 Effectiveness of Condensed memory

To examine whether our Condensed memory, which retains only the key rules that directly affect the final decision, can achieve comparable or better performance at lower cost, we compare four memory settings under the same GA framework with GPT-5.4. The other three settings are (1) No-Memory, which provides no SOP or external memory and relies only on the task input and tool descriptions, (2) Full-Memory, which uses the complete SOP as external knowledge, and (3) Redundant-Memory, which extends Condensed memory with background descriptions, definitions, and weakly relevant information in order to test whether extra content brings additional benefit. The evaluation is conducted on the dangerous goods subset of SOP-Bench [6]. We use Task Success Rate (TSR) to measure behavioral performance and Memory Size to measure contextual cost, and we further compare the performance gain achieved per unit of token cost across different memory settings.

As shown in Table 4, No-Memory performed substantially worse than all memory based settings, which shows that external procedural memory is necessary for this task. Condensed memory achieved the same highest TSR as Redundant-Memory and outperformed Full-Memory while using far fewer tokens, which indicates that **filtered high density memory is more effective than the original full SOP.** Redundant-Memory provided no gain over Condensed memory despite using more tokens, which suggests that background descriptions,

Table 4 Memory ablation on SOP-Bench dangerous_goods. Condensed memory achieves the best cost-effectiveness.

Configuration	Memory Size (tokens)	TSR (%)
No-Memory	0	13.87
Full-Memory	575	52.44
Redundant-Memory	288	66.48
Condensed memory	165	**66.48**

Table 5 Long-term factual memory evaluation on LoCoMo. GA outperforms embedding-based systems without using external retrieval.

	Multi-Hop		Temporal		Open-Domain		Single-Hop	
System	F1	BLEU	F1	BLEU	F1	BLEU	F1	BLEU
Mem0	39.32	28.43	50.03	43.33	18.32	13.80	40.32	32.33
A-MEM	29.03	20.48	46.83	38.84	13.11	12.94	44.68	37.01
OpenClaw	21.43	18.41	22.56	20.43	9.56	9.03	23.44	24.21
GA	**43.33**	**39.96**	**52.23**	**51.11**	**20.41**	**15.31**	**45.69**	**40.66**

definitions, and explanatory text add contextual burden without contributing additional behavioral value. Overall, this experiment shows that GA benefits most from memory that stores only the small set of task critical information the model does not already know.

3.3.3 Long-Term Fact Retention

This experiment examines whether GA has long term factual memory and whether it can remain competitive on factual recall and multi hop reasoning tasks without relying on an additional embedding model or a vector database. The compared methods include embedding based approaches, namely Mem0 [9] and A-MEM, and non embedding approaches, namely OpenClaw and GA. All methods use gpt5.4, and for the embedding based methods we adopt text-embedding-3-small as the embedding model. The evaluation is conducted on the first subset of LoCoMo [10], with Category 5 summary tasks removed to avoid interference from summarization ability. We use F1 to measure the correctness and completeness of factual content, and BLEU-1 to measure the lexical similarity between the generated answer and the reference answer.

As shown in Table 5, GA achieved the best F1 and BLEU-1 scores across all four task categories, which **indicates that its memory mechanism supports both accurate factual recall and stable answer generation**. Its advantage was especially clear on Multi-Hop and Temporal tasks, which suggests that GA can not only retain long term facts but also use them effectively in reasoning across time and across fact chains. Even in Open-Domain, which was the most difficult category for all methods, GA still achieved the highest score, and this shows that its memory organization remains effective even when structural cues are weak. Overall, these results show that long term factual memory can be achieved through precise memory organization and filtering, and that GA does not need an additional embedding model or a vector database to build this capability.

3.3.4 Context Explosion Prevention

To evaluate whether GA can prevent memory explosion under long term use and continuous skill expansion, we compare GA, Claude Code, CodeX, and OpenClaw after installing 20 skills for each agent and using them intensively over time. We then issue a minimal request "Hello" and measure the Full prompt length of each agent to assess how well its memory system controls context growth after prolonged use.

As shown in Table 6, the Full prompt length of GA is far lower than that of Claude Code, CodeX, and OpenClaw, which shows that GA can keep context growth under control even after long term use and continuous skill expansion. This advantage comes from its hierarchical memory management, where **memory**

Table 6 Full prompt length after installing 20 skills and intensive usage, measured on a minimal input. GA prevents context explosion.

System	Full Prompt Length (tokens)
Claude Code	22,821
CodeX	23,932
OpenClaw	43,321
GA	**2,298**

Table 7 Nine-round evolution trajectory on the LangChain GitHub research task. The experiment starts from scratch in Round #1, iteratively refines a textual SOP in Rounds #2–#5, and enters the codified execution regime in Rounds #6–#9.

Round	Stage	Time	LLM Calls	Input	Output	Cache Create	Cache Read	Total
#1	Initial run	7m30s	32	15,581	7,647	15,600	183,375	222,203
#2	SOP optimization	4m19s	12	5,045	4,860	4,553	51,883	66,341
#3	SOP optimization	2m53s	8	2,538	4,598	3,155	39,534	49,825
#4	SOP optimization	2m29s	9	3,265	3,682	3,435	41,376	51,758
#5	SOP optimization	2m50s	7	2,568	3,276	2,424	27,268	35,536
#6	Codified SOP	2m24s	6	1,855	1,064	1,930	20,913	25,762
#7	Codified SOP	1m41s	5	1,931	1,126	1,708	18,249	23,014
#8	Codified SOP	1m35s	5	1,884	990	1,586	18,229	22,689
#9	Codified SOP	1m38s	5	1,323	994	1,659	19,034	23,010

is not directly injected into the context but is retrieved only when the model actually needs it. Overall, the results show that GA can preserve long term memory capability without causing uncontrolled context expansion.

3.4 Self-Evolution Capability

3.4.1 Setup

Task and Setting. We evaluate GA's self-evolution process in two complementary settings. The first is a nine-round longitudinal study where GA repeatedly performs similar GitHub research tasks. Each round operates on a different task instance, allowing us to observe how accumulated experience translates into efficiency gains across structurally similar workflows. The second is an eight-task web benchmark used to test whether the same efficiency gains transfer across task types. The longitudinal study reveals the full trajectory from first execution to convergence, while the benchmark study tests whether the same pattern generalizes beyond a single workflow.

Evolution Stages. We summarize GA's self-evolution into three stages. **Stage 1: Natural-language execution**, where GA solves the task mainly through in-context reasoning and trial-and-error interaction. **Stage 2: SOP distillation**, where accumulated experience is compressed into a textual SOP. **Stage 3: Code-based execution**, where the verified workflow is further converted into executable logic.

3.4.2 Iteration-Wise Efficiency Trajectory

The nine-round trajectory shows a phase transition from high-entropy exploration to a stable low-cost execution regime. As shown in Table 7, relative to Round #1, the final execution in Round #9 reduces runtime from 7m30s to 1m38s, a reduction of 78.2%, reduces LLM calls from 32 to 5, a reduction of 84.4%, and reduces total tokens from 222,203 to 23,010, a reduction of 89.6%. The key pattern is not just the large first-to-last gap, but the structure of the transition itself. During the textual SOP stage, token cost decreases overall from 66k to 36k, despite minor fluctuations, which indicates that SOP compression removes

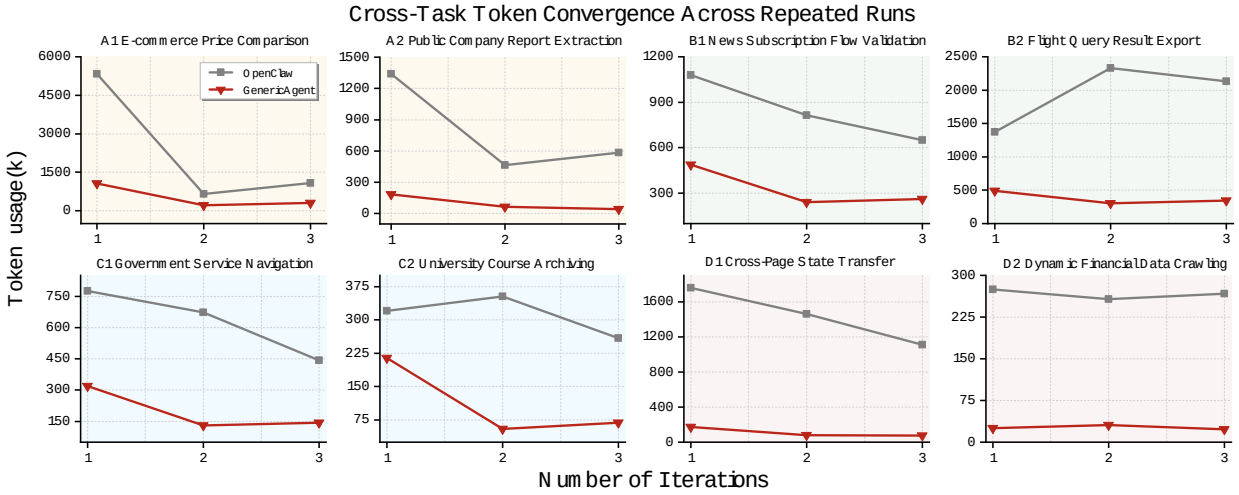

Figure 4 Cross-task token convergence across repeated runs for OpenClaw and GA. Each panel corresponds to one benchmark task and plots total token consumption over the first, second, and third executions.

most redundant exploration while still incurring a limited adaptation cost when the stored procedure must be aligned with newly encountered edge cases. Once the workflow is codified, the system enters a narrow convergence band of roughly 23k±1k tokens across Rounds #6–#9, indicating that execution has become predictable rather than exploratory.

Most of the efficiency gain comes from eliminating repeated reasoning cycles rather than merely shortening individual responses. The dominant change is the collapse in call count, which removes entire understand–reason–generate loops from the execution trace. This is consistent with the token composition: cache-read tokens fall from 183,375 in Round #1 to 19,034 in Round #9, and input tokens fall from 15,581 to 1,323, showing that the agent no longer rebuilds large amounts of context at runtime. Per-call token load also drops substantially over the course of evolution, from 6,944 tok/call in Round #1 to about 4.5k–4.6k tok/call in Rounds #7–#9, but this effect is secondary to the structural reduction in the number of calls. The larger implication is that textual SOPs already compress the search space, while code-level execution removes the remaining interpretation overhead and turns the learned workflow into an executable module.

3.4.3 Cross-Task Efficiency Gains

GA's self-evolving memory mechanism consistently improves efficiency across tasks rather than only helping on a single hand-optimized workflow. As shown in Figure 4, on all eight benchmark tasks, later GA executions consume fewer tokens than the first execution, with savings ranging from 61.0% to 92.4% and an overall reduction of 79.3%. This consistency matters more than any individual best case: it shows that SOP self-evolution is not just memorizing one task, but systematically converting prior successful trajectories into reusable execution shortcuts. The strongest gains appear in long-horizon tasks with state transfer and recovery (Category D, 92.0%), where repeated failure handling and path search dominate the initial execution cost. In those cases, SOP acts as path compression: once the verified route is stored, the agent can bypass most exploratory branches entirely.

GA exhibits a clear transition from cold start to rapid convergence across repeated executions of the same task. Across the three runs of each benchmark task, GA shows the same internal trend: the first run has substantially higher token cost, while the second and third runs quickly converge to a stable low-cost regime. This indicates that even with SOP memory, the first execution still incurs an SOP adaptation cost, because the agent must map generalized SOP knowledge onto the concrete page structure and interaction flow of the current task instance. However, this adaptation happens only once. After the successful path is aligned with the environment, later executions can directly reuse that result and rapidly drop to a stable token level. By contrast, OpenClaw shows no comparable convergence pattern across its three runs. On B2, for example,

Table 8 Web browsing evaluation results across three benchmarks. GA and OpenClaw both use claude-opus-4-6 as the backbone model.

Benchmark	Tasks	Evaluation	Score (0–1)		Avg. Tokens (M)	
			GA	OpenClaw	GA	OpenClaw
WebCanvas	12	Automatic	**0.834**	0.722	**0.18**	0.71
BrowseComp-ZH	10	LLM Judge	**0.60**	0.20	**0.47**	1.31
Custom Tasks	22	Human + LLM	**0.577**	0.50	**0.26**	0.76

token usage changes from 1,370k to 2,330k to 2,130k, which means the system keeps re-exploring rather than reusing prior experience. This distinction is important for deployment: GA pays a one-time adaptation cost on a new task, but its marginal cost quickly approaches zero, whereas the baseline remains expensive on every execution.

The benefit of SOP memory tends to increase with task complexity. When the tasks are ordered by OpenClaw's average token consumption as a proxy for intrinsic task complexity, a positive relationship emerges between complexity and saving rate. High-complexity tasks with OC averages above 1,000k tokens, such as A1, B2, and D1, achieve an average saving of 83.5%, while mid-complexity tasks average 72.5%. Although D2 is low in absolute token cost, its 90.1% saving rate is consistent with the same mechanism because it still contains long-horizon dependency structure. The broader implication is that SOP self-evolution becomes more valuable as tasks require more multi-step reasoning, navigation, and recovery from branching execution paths: in simple tasks it is helpful, but in complex tasks it becomes structurally decisive.

3.5 Web Browsing Capability

3.5.1 Setup

We evaluate GA's web browsing capability on three benchmarks. **(1) WebCanvas** [11]: 12 tasks measuring basic browser interaction such as navigation, clicking, filtering, and information extraction. **(2) BrowseComp-ZH** [12]: 10 tasks measuring multi hop web search and chain based reasoning in the Chinese web setting. **(3) Custom Tasks:** 22 real world tasks covering academic platforms, social media, content sites, utility websites, e commerce platforms, and public retrieval services, scored on a 0 to 3 scale through manual review with GPT-5.4 assisted judgment. Both GA and OpenClaw use claude-opus-4-6 as the backbone model.

3.5.2 Results

As shown in Table 8, GA outperforms OpenClaw across all three benchmarks while consuming significantly fewer tokens. On WebCanvas, GA achieves 0.834 score with 0.18M tokens versus OpenClaw's 0.722 score with 0.71M tokens. On BrowseComp-ZH, GA reaches 0.60 accuracy compared to OpenClaw's 0.20, using 0.47M versus 1.31M tokens. On Custom Tasks, GA scores 0.577 versus 0.50, winning 10 tasks against 6, while using only 0.26M tokens compared to 0.76M for OpenClaw.

GA demonstrates consistent token efficiency across diverse task types, reducing token consumption by 2.9x to 3.9x while maintaining competitive or superior performance. The performance gap is particularly pronounced on BrowseComp-ZH, where GA's multi-stage compression pipeline preserves context density, enabling deeper reasoning chains without context overflow. This combination of efficiency and effectiveness makes GA practical for real-world web automation scenarios where token cost and latency are critical constraints. A detailed visualization is provided in Appendix 2.

4 Related Work

Prior work on autonomous LLM agents has advanced several ingredients of long-horizon execution, including action interfaces, memory, self-improvement, and web interaction. Our position relative to this literature is more specific than a generic "integrated system" claim. GA is built around a single systems objective, *maximizing contextual information density*, and studies how multiple agent components should be co-designed

when the bottleneck is not raw capability but how much decision-relevant information can be preserved within a limited context budget. In this sense, GA is not simply a new memory module, a new reflection method, or a new browser agent; it is a general-purpose agent architecture that connects these ingredients through a common optimization target.

4.1 LLM-Based Agent Systems and Action Interfaces

LLM agents have evolved from prompt-level reasoning loops to integrated systems that execute long action sequences in real environments. ReAct [13] and Reflexion [14] established the basic loop of interleaving reasoning, acting, and feedback. AutoGPT [15] popularized iterative goal decomposition, while MetaGPT [16] pushed the field toward explicit multi-role coordination and workflow design. This line established the key intuition that agent performance depends not only on the base model, but also on how deliberation is scaffolded over multiple steps.

As agents moved into software engineering and computer-use settings, the action interface itself became a first-class design choice. CodeAct [17] unifies agent actions as executable code, which increases compositional flexibility and makes behaviors easier to test and reuse. Devin [18], SWE-agent [19], and OpenHands [20] further show that performance depends strongly on how the model is connected to the external environment, whether through integrated coding workflows, specialized Agent–Computer Interfaces, or open agent runtimes. Recent product systems such as Claude Code [1], Codex [21], Manus [22], and OpenClaw [2] reinforce the same trend in practice.

GA is closest to this systems line, but differs in what it optimizes. Most existing agent systems primarily expand reachable behaviors through richer workflows, more specialized tools, or stronger environment integration. By contrast, GA asks how broad computer-use capability can be preserved while keeping the action space and prompt overhead deliberately small. Its minimal atomic toolset is therefore not just an implementation choice, but part of the paper's central claim: cross-domain long-horizon performance can improve when tool abstraction is designed to increase contextual information density rather than to maximize interface richness.

4.2 Memory and Context Management

A second line of work studies how agents retain only behaviorally useful information as trajectories grow. MemGPT [23] treats the context window as a limited working memory and external storage as archival memory, introducing a paging-style view of agent memory. A-MEM [24] instead models memory as a dynamically evolving network of atomic notes and links, enabling richer associative recall. These approaches make long-horizon agents more feasible, but they focus primarily on storage and retrieval.

Recent work increasingly treats context construction itself as a systems problem. LongLLMLingua [25] shows that prompt compression can preserve task-relevant information while reducing long-context cost, and practical context-engineering analyses emphasize that agent quality depends not only on window length but also on what enters the window and in what form [5]. Production systems such as Claude Code [1] and Manus [22] similarly rely on artifact-based tracking and periodic compaction to extend effective horizon.

GA differs from both memory-centric and compression-centric approaches in two ways. First, it treats memory quality as a *verification and selection* problem, not only a storage or retrieval problem: only behavior-changing and validated information is promoted into longer-term representations. Second, it optimizes the full path from observation to retained memory, rather than only the final retrieval step. The resulting design goal is not to keep more history accessible, but to keep the active context as sparse, reliable, and decision-relevant as possible.

4.3 Self-Evolution and Experience Distillation

Research on self-evolving agents asks how repeated execution can become future capability. A recent survey [26] organizes the space by what evolves, when evolution happens, and what feedback drives improvement. Within this space, Agent-Pro [27] studies policy-level reflection and optimization, showing that agents can revise their own behavioral policies without updating model parameters. Voyager [28] demonstrates a stronger form of accumulation in a specialized environment by continually storing verified executable skills.

Broader general-purpose work mostly evolves the agent through textual abstractions. EvolveR [29], FLEX [30], AgentEvolver [31], and experience-driven lifelong learning [32] all convert trajectories into strategic principles, reflections, or structured knowledge units that help later execution. This is an important step beyond simple history reuse, but in most cases the retained experience remains natural-language guidance rather than executable capability.

GA is aligned with this line in viewing reflection as the engine of continual improvement, but it makes a stronger systems claim: the endpoint of reflection should be a *representation shift* from verbose trajectories to compact operational assets. Verified experience is progressively transformed into SOPs, code, and reusable skills, so improvement appears not only as better policy guidance but also as lower inference-time cost. This is why our evaluation emphasizes efficiency convergence and token reduction under repeated execution, rather than only one-shot task success.

5 Conclusion

We present GenericAgent, a general-purpose LLM agent system built around context information density maximization. The central idea is that long-horizon performance depends not only on model capability, but also on how efficiently decision-relevant information is maintained in context. GA operationalizes this principle through four components: a minimal atomic tool set that reduces system overhead, a hierarchical memory that retains only verified and useful information, reflection-driven self-evolution that distills trajectories into reusable skills, and structured browser extraction that compresses noisy web pages into task-relevant observations. Experiments show that these components jointly improve performance and efficiency. GA achieves strong task completion with a better completion–cost trade-off than representative baselines. Its efficiency improves over repeated execution, indicating that experience is distilled into reusable capability rather than stored as raw history. The memory system supports long-term factual recall without external vector retrieval and mitigates context explosion in prolonged interactions. The browser module further improves robustness and efficiency on real-world web tasks.

References

[1] Anthropic. Claude code. https://www.anthropic.com/product/claude-code, 2025.

[2] Xingyao Wang et al. Openclaw-rl: Training agents via interaction. *arXiv preprint arXiv:2603.10165*, 2026.

[3] Lost in the middle: How language models use long contexts. https://arxiv.org/abs/2307.03172, 2023.

[4] Llms get lost in multi-turn conversation. https://arxiv.org/abs/2505.06120, 2025.

[5] Aditya Rajasekaran et al. Effective context engineering for ai agents. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, September 2025.

[6] Subhrangshu Nandi, Arghya Datta, Rohith Nama, Udita Patel, Nikhil Vichare, Indranil Bhattacharya, Prince Grover, Shivam Asija, Giuseppe Carenini, Wei Zhang, Arushi Gupta, Sreyoshi Bhaduri, Jing Xu, Huzefa Raja, Shayan Ray, Aaron Chan, Esther Xu Fei, Gaoyuan Du, Zuhaib Akhtar, Harshita Asnani, Weian Chan, Ming Xiong, Francesco Carbone, and Jeetu Mirchandani. Sop-bench: Complex industrial sops for evaluating llm agents. *arXiv preprint arXiv:2506.08119*, 2025.

[7] Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, ZhongZhi Li, Yingying Zhang, Le Song, and Qianli Ma. Lifelongagentbench: Evaluating llm agents as lifelong learners. *arXiv preprint arXiv:2505.11942*, 2025.

[8] Yuyang Dai, Yan Lin, Zhuohan Xie, and Yuxia Wang. Realfin: How well do llms reason about finance when users leave things unsaid? *arXiv preprint arXiv:2602.07096*, 2026.

[9] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[10] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024.

[11] Yichen Pan, Dehan Kong, Sida Zhou, Cheng Cui, Yifei Leng, Bing Jiang, Hangyu Liu, Yanyi Shang, Shuyan Zhou, Tongshuang Wu, and Zhengyang Wu. Webcanvas: Benchmarking web agents in online environments. *arXiv preprint arXiv:2406.12373*, 2024.

[12] Peilin Zhou, Bruce Leon, Xiang Ying, Can Zhang, Yifan Shao, Qichen Ye, Dading Chong, Zhiling Jin, Chenxuan Xie, Meng Cao, Yuxin Gu, Sixin Hong, Jing Ren, Jian Chen, Chao Liu, and Yining Hua. Browsecomp-zh: Benchmarking web browsing ability of large language models in chinese. *arXiv preprint arXiv:2504.19314*, 2025.

[13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[14] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.

[15] Significant Gravitas. Autogpt. https://github.com/Significant-Gravitas/AutoGPT, 2023.

[16] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.

[17] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. *arXiv preprint arXiv:2402.01030*, 2024. Accepted at ICML 2024.

[18] Cognition Labs. Devin: Ai software engineer. https://cognition.ai/blog/introducing-devin, 2024.

[19] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Liber, Shunyu Yao Narasimhan, and Karthik. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37, 2024.

[20] Xingyao Wang, Boxuan Jiang, Ziniu Lu, Yufan Liu, Abishek Sridhar Li, Bolun Shi, Jiannan Fang, Rithvik Mohanty, Niklas Muennighoff Ponnapalli, Kaixuan Ren, et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, 2025.

[21] OpenAI. Introducing codex. https://openai.com/index/introducing-codex/, 2025.

[22] Yichao Shen et al. Manus: From mind to machine. *arXiv preprint arXiv:2505.02024*, 2025.

[23] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[24] Wujiang Xu et al. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38, 2025.

[25] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.

[26] Zhiyuan Gao et al. A survey of self-evolving agents. *Transactions on Machine Learning Research*, 2026.

[27] Wenqi Zhang, Ke Tang, Hai Wu, Mengna Wang, Yongliang Shen, Guiyang Hou, Zeqi Tan, Peng Li, Yueting Zhuang, and Weiming Lu. Agent-pro: Learning to evolve via policy-level reflection and optimization. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5348–5375, 2024.

[28] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[29] Xuechen Wu et al. Evolver: Self-evolving llm agents through experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*, 2025.

[30] Anonymous. Flex: Continuous agent evolution via forward learning from experience. *arXiv preprint arXiv:2511.06449*, 2025.

[31] Anonymous. Agentevolver: Towards efficient self-evolving agent system. *arXiv preprint arXiv:2511.10395*, 2025.

[32] Tianyu Cai et al. Building self-evolving agents via experience-driven lifelong learning. *arXiv preprint arXiv:2508.19005*, 2025.

6 Author Contributions

This project is led by **Jiaqing Liang** and **Yanghua Xiao**. The development of the GA and the preparation of this manuscript involve contributions from multiple authors across different aspects, including system design, experimentation, and writing. Below we detail the specific contributions of each author.

Jiaqing Liang: Led core system design, implemented the main codebase, and was responsible for project management.

Jinyi Han: Led paper organization and task allocation, oversaw the manuscript preparation from initial draft to final version, and wrote Sections 1, 2 and 5.

Weijia Li: Conducted evaluation experiments for Section 3.5 and contributed to the writing and refinement of Section 3.5.

Xinyi Wang: Conducted evaluation experiments for Section 3.3 , contributed to the writing of Sections 3.3 and 3.5.

Zhoujia Zhang: Conducted evaluation experiments for Section 3.4.

Zishang Jiang: Wrote Section 3.1, 3.2 and 3.2.

Ying Liao: Conducted evaluation experiments for Section 3.2 and generated figures for Section 3.

Tingyun Li: Wrote Section 4 and the Appendix.

Ying Huang: Conducted evaluation experiments on RealFineBench for Section 3.1.

Keyi Wang: Conducted evaluation experiments on SOP-Bench for Section 3.1.

Zhonghua Hong: Conducted evaluation experiments on Lifelong AgentBench for Section 3.1.

Zhiyu Lu: Performed partial evaluation experiments for models in Section 3.1.

Lipeng Ma: Provided supervision for evaluation experiments in Section 3.1.

Sihang Jiang: Provided supervision for evaluation experiments in Section 3.2.

Yanghua Xiao: Provided overall project supervision and strategic guidance.

1 Atomic Tool Alignment

This subsection provides a detailed capability-level alignment for the core atomic tools discussed in the main text. The purpose is not to enumerate the full runtime tool inventories of Claude Code or OpenClaw, but to show that the core capabilities retained by GA each have corresponding prototypes in both systems.

Table 9 Capability-level alignment of the basic tooling environment. Each row maps one capability category to the corresponding tool in Claude Code, OpenClaw, and GA.

Capability Category	Corresponding Tool / Mechanism		
	Claude Code	**OpenClaw**	**GA**
Code execution	BashTool	exec / process / nodes	code_run
File reading	FileReadTool	read	file_read
File writing	FileWriteTool	write	file_write
Local file editing	FileEditTool	write / patch-like editing	file_patch
Web reading	WebFetchTool / browser read	browser base reading	web_scan
Web interaction	WebBrowserTool / browser actions	browser interaction	web_execute_js
Working memory	TodoWriteTool / BriefTool	session history / status-like ability	update_working_checkpoint

Table 10 complements the capability-level mapping above with representative task-level replacement examples. Rather than claiming that GA reproduces every specialized tool one by one, the point is that many benchmarked capabilities can be reconstructed through short compositions of a few atomic tools.

Source	Specialized Tool / Capability	Representative Task	GA Replacement Composition	Argument
Claude Code	GlobTool	task_cc_01_glob_-markdown_list	code_run	File-pattern matching can be completed through general code execution.
Claude Code	GrepTool	task_cc_02_grep_-todo_hits	code_run + file_read	Text search and localization can be implemented by combining general execution with file reading.
Claude Code	WebFetchTool	task_cc_03_webfetch_-example_brief	web_scan + web_execute_js + code_run	Web fetching, cleaning, and summarization can be reconstructed by combining browser primitives with code-based post-processing.
Claude Code	AgentTool	task_cc_04_agent_-fact_extract	file_read + code_run + update_working_-checkpoint	Explicit agent delegation can be approximated through stored subagent strategies in memory together with general execution triggers.
Claude Code	WebSearchTool	task_cc_05_-websearch_example_-results	web_execute_js + web_scan	Online search can be completed through browser interaction plus webpage reading.
Claude Code	NotebookEditTool	task_cc_06_notebook_-inspect	file_read + code_run	Notebook-structure inspection can be handled by reading the .ipynb JSON and executing a parsing script.
OpenClaw	nodes / structured data	task_oc_02_csv_stats	code_run	CSV statistics can be completed through general code execution without a dedicated structured-data tool.

Table 10 Representative task-level substitutions from specialized tools to GA atomic-tool compositions. Each row shows one concrete capability test, the original specialized tool or mechanism used by the baseline system, and the corresponding GA composition that can complete the same class of task.

2 Web Browsing Visualization

Figure 5 provides a visual comparison of token consumption and normalized performance across the three web browsing benchmarks discussed in Section 3.5.

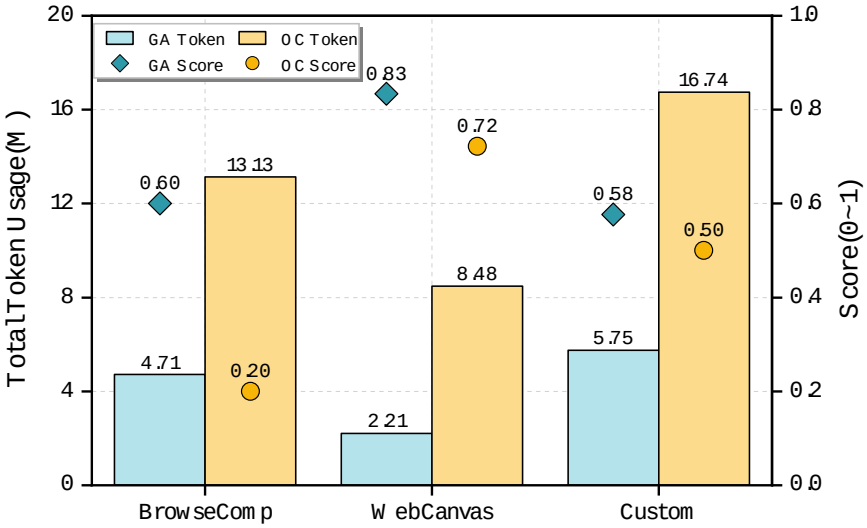

Figure 5 Token consumption and normalized score across three web browsing benchmarks. The left axis shows total token consumption in millions (M), while the right axis shows normalized scores on a 0–1 scale. GA consistently achieves competitive or superior performance while consuming significantly fewer tokens than OpenClaw across all three benchmarks.

3 Case Studies

We present four representative cases corresponding to the main evaluation dimensions. For readability, the cases are ordered as tool use, memory, self-evolution, and web browsing. Each case follows a "purpose and task setup → artifacts / trace → observed outcome" pattern so that the appendix reads as a set of worked examples rather than a list of compressed summaries.

- **Case A1 (Tool Use).** An API procurement workflow requiring web lookup, structured extraction, budget reasoning, and file generation, used to show how GA handles a long-horizon task with a minimal atomic-tool set (Section 3.2).
- **Case A2 (Memory).** Dangerous-goods hazard classification under condensed, full, and redundant memory variants, used to show why information density matters more than information volume (Section 3.3).
- **Case A3 (Self-Evolution).** A GitHub PR research task showing the progression from natural-language SOP to compiled Python code and a transferability note, used to demonstrate how GA converts experience into reusable executable assets (Section 3.4).
- **Case A4 (Web Browsing).** A BrowseComp-ZH multi-hop reasoning chain where GA identifies a historical figure through iterative search refinement, used to illustrate structured browser extraction in practice (Section 3.5).

3.1 Case A1. Tool Use: API Procurement Workflow

Purpose and Task Setup

```
Purpose: to examine whether GA's atomic-tool design can support a long-horizon, multi-step
workflow,
we provide a laboratory procurement task over public LLM pricing pages.
This task is useful because it requires browsing, structured extraction, numerical reasoning,
and file generation in one workflow.
The goal of the case is to show that GA's minimal atomic-tool set can complete the full chain
and reach the correct recommendation, providing a qualitative counterpart to Table 11.
Task: investigate laboratory procurement for text-only LLM APIs using these public pricing
pages:
- OpenAI API pricing: https://openai.com/api/pricing/
- Anthropic pricing: https://www.anthropic.com/pricing
- Gemini API pricing: https://ai.google.dev/gemini-api/docs/pricing
Constraints:
- only text models are considered
- laboratory size: 6 students
- monthly input tokens: 80M
- monthly output tokens: 20M
- budget: $300 / month
Requirements:
1. record candidate flagship text models and their input/output prices
2. test whether a single-model plan fits the budget
3. if not, trigger fallback and test a dual-model plan
4. if that still fails, output infeasible
5. final answer must state the primary option, whether it is feasible, whether fallback was
triggered, the recommended plan, the estimated monthly cost, and the reason
Required output files: cost_comparison.csv and decision_04.json.
```

Artifact 1: Cost Comparison File

```
cost_comparison.csv
provider,model,input_cost_per_1m,output_cost_per_1m,estimated_monthly_cost_usd,meets_budget
OpenAI, GPT-5.4, 2.5, 15.0, 500.0, No
OpenAI, GPT-5.4 mini, 0.75, 4.5, 150.0, Yes
OpenAI, GPT-5.4 nano, 0.2, 1.25, 41.0, Yes
Anthropic, Claude Opus 4.6, 5.0, 25.0, 900.0, No
Anthropic, Claude Sonnet 4.6, 3.0, 15.0, 540.0, No
Anthropic, Claude Haiku 4.5, 1.0, 5.0, 180.0, Yes
Google, Gemini 2.5 Pro, 1.25, 10.0, 300.0, Yes
Google, Gemini 2.5 Flash, 0.3, 2.5, 74.0, Yes
Google, Gemini 2.5 Flash-Lite, 0.1, 0.4, 16.0, Yes
```

Artifact 2: Decision File

```
primary_option: Gemini 2.5 Pro
primary_option_feasible: true
fallback_triggered: false
recommended_plan: single_model
recommended_models: [Gemini 2.5 Pro]
estimated_monthly_cost_usd: 300.0
reason: Gemini 2.5 Pro is a strong single-model option that still fits within the budget.
Cost calculation: 80M input x $1.25/MTok + 20M output x $10.00/MTok = $300.0/month.
Other flagship models exceed the budget: GPT-5.4 at $500/month and Claude Opus 4.6 at
$900/month.
```

Table 11 Cross-system comparison on the long-horizon procurement task.

System	Requests	Total Tokens	Time (s)	Success
GenericAgent	21	364,385	324.64	Yes
Claude Code	29	485,335	224.53	Yes
OpenClaw	12	428,402	108.42	Yes

```
No fallback to a dual-model plan is needed.
```

Observed Outcome

```
GA completed the task in 21 requests and 364,385 total tokens (324.6s).
All grading criteria passed: cost_comparison.csv and decision_04.json both created with
correct schema.
Final recommendation: Gemini 2.5 Pro as a single-model plan at exactly $300/month.
Cost breakdown: 80M input × $1.25/MTok + 20M output × $10.00/MTok = $300.0.
Other flagship models exceeded the budget: GPT-5.4 at $500/month, Claude Opus 4.6 at
$900/month.
No fallback to a dual-model plan was needed.
```

3.2 Case A2. Memory: Dangerous-Goods Hazard Classification

Purpose and Task Setup

```
Purpose: to examine whether memory quality depends more on information density than on raw
information volume,
we provide a dangerous-goods classification task under three memory variants.
This task is useful because the decision rule stays fixed while only the memory
representation changes.
The goal of the case is to show that the main effect comes from how the rules are packaged
and retrieved, rather than from whether the rules exist at all.
Task: determine hazard_class from product_id and the four source texts.
Inputs:
- SDS label text
- handling/storage guidance
- transportation requirements
- disposal guidance
The output must contain both hazard_score and hazard_class.
```

Artifact 1: Condensed Memory

```
Required rules:
1. First validate product_id. It must be resolvable and follow format P_XXXXX.
If invalid, stop and output hazard_score=0 and hazard_class="Unable to Decide".
2. Get four component scores:
- safety score from SDS label text
- handling score from handling/storage guidelines
- transportation score from transportation requirements
- disposal score from disposal guidelines
Each valid component score must be in [1,5].
3. Compute hazard_score:
- If one component is missing or 0, replace it with the max of the other available scores.
- If more than two components are missing or 0, output hazard_score=0 and hazard_-
```

```
class="Unable to Decide".
- Otherwise hazard_score = sum of the four component scores.
- Final valid range is 4 to 20.
4.  Map hazard_score to class:
- 4-7 to Hazard Class A
- 8-12 to Hazard Class B
- 13-16 to Hazard Class C
- 17-20 to Hazard Class D
5.  Final output must include both hazard_score and hazard_class.
```

Artifact 2: Full Memory

```
1.  Purpose
To establish a standardized methodology for the systematic identification and classification
of dangerous goods hazard classes through multi-source data integration and quantitative
severity assessment protocols.
2.  Scope
This procedure encompasses all dangerous goods shipment classification processes within the
organization's supply chain operations.
3.  Definitions
SDS = Safety Data Sheet
HS = Handling and Storage Guidelines
TR = Transportation Requirements
DG = Disposal Guidelines
AIP = API Integration Protocol
HCM = Hazard Classification Matrix
SAS = Severity Assessment Score
4.  Input
Product ID (format:  P_XXXXX)
Source documentation:  SDS / Handling and Storage / Transportation / Disposal
API access credentials:  endpoint URLs / authentication tokens / backup authentication
protocols
5.  Main Procedure
Validate product identification documentation completeness.
If product ID fails format requirements, no further action should be taken.
The hazard score should be marked as 0 and hazard class as Unable to Decide.
Extract four scores, each between 1 and 5.
hazard_score = safety score + handling score + transportation score + disposal score
If any component is missing or 0, impute it by taking the max of the other scores.
If more than two component scores are missing, mark Unable to Decide.
6.  Output
Final hazard class designation
Digital record in Hazard Classification Registry
API response logs for all scoring calculations
Classification audit trail documentation
Final output should be in XML format with tags <hazard_score> and <hazard_class>
```

Artifact 3: Redundant Memory

```
You are following a standardized dangerous-goods classification procedure designed for
supply-chain compliance and multi-source hazard assessment.
Definitions:
- SDS = safety data sheet
- HS = handling and storage guidance
- TR = transportation requirements
- DG = disposal guidelines
```

```
- SAS = severity assessment score from 1 to 5
- HCM = hazard classification matrix
Input scope:  product_id, SDS text, handling/storage text, transportation text, disposal
text, API access and validation context.
Decision workflow:
1.  Validate documentation completeness and product identifier validity.
2.  product_id must match format P_XXXXX and be resolvable.
3.  If invalid, do not continue.  Set hazard_score to 0 and hazard_class to "Unable to
Decide".
4.  Analyze SDS, handling/storage, transportation, and disposal sources separately.
5.  Derive one component score per source, each between 1 and 5.
6.  If exactly one component is missing or 0, impute it using the maximum of the other
component scores.
7.  If more than two components are missing, return "Unable to Decide" and hazard_score 0.
8.  Sum the four components into a cumulative hazard_score and validate that total score lies
in the range 4-20.
9.  Convert the total score into Hazard Class A / B / C / D.
10.  Return the final result in structured form with hazard_score and hazard_class,
preserving an audit-ready trail.
Background note:  the broader SOP also mentions registry logging, API integration,
source-document handling, and record retention, but the operationally decisive rules are
the identifier check, missing-value handling, score summation, and class mapping.
```

Observed Outcome

The memory ablation becomes visually intuitive once the three variants are shown side by side. The condensed file places every behavior-changing rule in a single short block, whereas the full file spreads the same operational core across purpose statements, definitions, credential notes, registry requirements, and output-format details. Table 4 can therefore be read primarily as an information-placement result rather than a simple information-volume result.

3.3 Case A3. Self-Evolution: GitHub PR Research

Purpose and Task Setup

```
Purpose:  to examine whether repeated execution can be distilled into reusable assets,
we provide a GitHub PR research task that evolves from an SOP into executable code.
This task is useful because the representation shift is directly observable:  the agent first
stabilizes a procedure as text and then compiles that procedure into code.
The goal of the case is to show that what changes across iterations is not merely token
count, but the form of reusable knowledge, which serves as the qualitative counterpart of
Table 7.
Investigate the five most recent merged bug-fix PRs in a GitHub repository.
For each PR: recover the linked issue, identify the affected module, and verify whether the
module has troubleshooting coverage in the documentation site.
Produce a structured JSON report.
```

Artifact 1: SOP

```
Scenario:  batch investigation of GitHub PRs, including PR details, linked issues, affected
modules, and documentation coverage.
Core strategy:
DO NOT browse PR pages one by one - efficiency is extremely low and the run may exceed the
turn budget.
PREFER batch extraction - use a script or API to obtain multiple PR records at once.
```

```
Recommended solution:  use github_pr_analyzer.py to complete the whole chain in one pass.
Example command:
python3 ../memory/github_pr_analyzer.py langchain-ai/langchain
-doc-url https://python.langchain.com
-limit 5
-output report.json
Script advantages:
- pure Python, no browser environment required
- one-click execution of the full workflow
- flexible command-line parameters
- built-in error handling
Manual fallback flow:
1.  build a filtered PR URL
2.  use document.querySelectorAll(.js-issue-row) to extract PR numbers and titles
3.  batch fetch PR HTML
4.  recover issue links with the rule /issues/\d+, then Fixes/Closes #\d+, then standalone
#\d+
5.  infer module names from PR title prefixes such as community:
6.  verify documentation coverage against troubleshooting and integration paths
7.  dump the final report with json.dump()
Pitfalls:
1.  do not use window.location.href to visit PR pages one by one
2.  prefer browser fetch() when manual HTML collection is needed
3.  distinguish issue links from pull-request links
4.  different projects may use different documentation subdomains
```

Artifact 2: Compiled Code

fetch_pr_list(...)

```
url = https://github.com/{repo}/pulls?q={filters}
response = session.get(url, timeout=30)
soup = BeautifulSoup(response.text, 'html.parser')
pr_elements = soup.select('.js-issue-row')
collect number, title, and url for the first limit PRs
```

extract_pr_details(pr)

```
module_match = re.match(r'^([^:\[]+)', pr['title'])
pattern 1:  full GitHub issue URL
pattern 2:  Fixes|Closes|Resolves #N
pattern 3:  standalone #N excluding the PR number itself
return pr_number, bug_module, issue_link
```

check_doc_coverage(module)

```
probe multiple candidate paths:
/docs/troubleshooting/
/docs/troubleshooting/{module}
/docs/integrations/{module}/
/docs/modules/{module}/
/api_reference/{module}/
/docs/integrations/providers/{module}/
accept the module if the page contains both the module name and troubleshooting
```

analyze(...)

```
fetch PR list, then extract details, then check documentation, then call json.dump(results,
f, indent=2, ensure_ascii=False)
```

Artifact 3: Transferability Note

```
Original task core abilities:
- batch retrieval of GitHub PR lists with filters
- extraction of PR details, titles, modules, and linked issues
- verification of external documentation coverage
- generation of structured JSON reports
Reusable asset set:
- SOP document: github_pr_research_sop.md
- Python script: github_pr_analyzer.py
Transfer targets described in the case note:
1. GitHub issue batch research
2. contributor analysis
3. release tracking
4. workflow and CI inspection
5. code search and pattern extraction
Example reuse in the issue-analysis case:
- keep the batch-fetch strategy
- replace /repos/{repo}/pulls with /repos/{repo}/issues
- reuse regex extraction and concurrent request handling
Estimated adaptation cost in the transferability note: about 30% code changes, mainly
endpoint replacement and field mapping.
```

Observed Outcome

```
LangChain verification case (2026-04-08):
python3 github_pr_analyzer.py langchain-ai/langchain -doc-url https://python.langchain.com
-limit 5

5 merged bug-fix PRs retrieved
modules found: community (2), partners, openai, core/anthropic
issue links recovered: 4/5
troubleshooting coverage found: 4/5
runtime: about 30-40 seconds
```

3.4 Case A4. Web Browsing: BrowseComp-ZH Reasoning Chain

Purpose and Task Setup

Purpose: to examine multi-hop web browsing under a compressed browser interface,
we provide a BrowseComp-ZH question whose answer must be recovered through iterative query refinement.
This example is useful because the answer is not available from a single lookup and the chain of evidence must be assembled step by step.
The goal of the case is to make the narrowing process visible: identify the *shihua* pioneer, confirm Ouyang Xiu, and then recover Wang Anshi from the quotation.
Original question (BrowseComp-ZH):
"There is a writer A who participated in a literary reform movement,
advocated a plain but disciplined style, opposed overly strange and risky writing,
and pioneered the *shihua* form of poetic criticism.
Another contemporary writer B commented that he had "deep character" and "broad knowledge".
Who was writer B?"
Recorded case metadata: dataset = browsecomp, score = 1.0, time = 130.4s, tokens = input 30050 / output 1296, turns = 7, tools = 6.

Artifact 1: Interaction Trace

```
Turn 1 summary: start by searching ‘‘first shihua writer’’ to identify writer A.
Action query: shihua inventor plain style
Turn 2 summary: scan the Google result page and narrow the candidate set.
Turn 3 summary: because the first search does not uniquely resolve the target, search
‘‘Ouyang Xiu + Liuyi Shihua’’.
Action query: Ouyang Xiu Liuyi Shihua
Turn 4 summary: confirm writer A as Ouyang Xiu.
Turn 5 summary: search the evaluative phrase together with Ouyang Xiu.
Action query: deep character broad knowledge Ouyang Xiu
Turn 6 summary: verify that the quoted evaluator is Wang Anshi.
Turn 7 summary: return the final answer with confidence.
```

Artifact 2: Final Reasoning Excerpt

```
Explanation:
Through the searches on ‘‘first shihua writer’’ and related keywords, the agent identified
writer A as Ouyang Xiu.
He participated in the Northern Song literary reform movement, advocated a plain but
disciplined style,
opposed strange and risky writing, and pioneered the form through Liuyi Shihua.
After searching the evaluative phrase together with Ouyang Xiu,
the agent found that Wang Anshi, in a commemorative text for Ouyang Xiu,
described him as having ‘‘deep character’’ and ‘‘broad knowledge’’.
Therefore the writer B in the question is Wang Anshi.
```

Observed Outcome

```
score: 1.0
turns: 7
tool calls: 6
time: 130.4s
Exact Answer: Wang Anshi
Judge result: correct = yes, confidence = 95%
Reasoning note: the extracted final answer exactly matches the gold answer ‘‘Wang Anshi’’.
```